



Sistema Inteligente de Gestión y Optimización de Rutas de Emergencia

ICC-708

Alumnos:

Juan Inzunza.

Carrera: Ingeniería Informática

1. Introducción y Contexto

Durante un desastre natural, los centros de emergencia reciben numerosas solicitudes de ayuda que deben priorizarse y atenderse con rutas eficientes. Ante esto, el presente informe documenta el diseño e implementación de un sistema de gestión de incidentes de emergencia desarrollado en Python, que integra estructuras de datos propias (tabla hash, cola de prioridad basada en heap), algoritmos de ordenamiento (MergeSort, QuickSort), un modelo de grafo para la “red vial”, y tres algoritmos de búsqueda de rutas (BFS, UCS y A* con heurística geográfica).

El sistema cuenta con una UI construida con Tkinter y Matplotlib lo que le permite cargar y visualizar la red y las rutas calculadas de manera más natural para el usuario.

Notar que el informe se centra en diseño y tiempos, pero no la lógica de todas las funciones como tal. Alargaría mucho el texto, por lo que se recomienda ver el código (que está comentado) para ver la lógica aplicada al 100%.

El código se encuentra en el siguiente repositorio:

<https://github.com/JJIRivas/Lab-prog-avanzada/tree/Proyecto-Final>

1.1 Objetivos cubiertos

- Registrar y gestionar incidentes mediante una tabla hash.
- Priorizar solicitudes de emergencia mediante una cola de prioridad.
- Generar resúmenes ordenados.
- Modelar la red como un grafo ponderado para calcular las rutas óptimas.

2. Modelado de Datos

Se definieron tres ADTs según lo especificado: Incident, EmergencyCenter y RoadNetwork.

2.1 Incident

Representa un incidente reportado. Es un ADT sin operaciones propias más allá de la construcción, ya que toda la lógica necesaria que se debe implementar vive en las estructuras que lo contienen (hashTable, pHeap).

Vista general de un Incidente:

Atributo	Tipo	Descripción
id	str	Identificador único, clave de la tabla hash
ubicación	str	Nodo de la red vial donde ocurrió el incidente
prioridad	int (1-5)	Severidad reportada por el usuario (1=min, 5=Max)
tipo	str	Categoría del incidente (incendio, accidente, etc.)
timestamp	datetime	Momento del reporte, usado para calcular urgencia
estado	str	pendiente / en atencion

Precondición: Todos los campos deben estar presentes y el timestamp debe ser un datetime válido (se parsea desde string con formato "%Y-%m-%d %H:%M:%S").

Postcondición: el objeto queda inmutable en su identidad (id) pero mutable en estado.

2.2 EmergencyCenter

Representa un centro de operaciones. Al igual que Incident, su única responsabilidad es asociar un identificador y nombre a una ubicación (nodo del grafo). Las lógicas necesitadas viven específicamente en urgencyManager.py, porque esa operación depende de tres otros factores a la vez (lista de centros, red vial como tal, y el algoritmo de búsqueda) y no es responsabilidad de un único centro.

Atributo	Tipo	Descripción
id	str	Identificador único del centro
nombre	str	Nombre para mostrar en la interfaz
ubicación	str	Nodo del grafo donde está ubicado el centro

2.3 RoadNetwork

Modela la red vial como un grafo ponderado dirigido. Internamente delega el almacenamiento del grafo a la estructura graph y la carga de la información relevante desde los archivos mediante nodeParser y edgeParser.

Atributo	Tipo	Descripción
id	str	Identificador único
grafo	graph.py	Grafo vacío.
node_parser	nodeParser.py	Parser para los Nodos (ubicaciones).
edge_parser	edgeParser.py	Parser para las aristas (caminos).

Operación	Descripción
cargarNodos(filepath)	Agrega X nodos con sus coordenadas
cargarAristas(filepath, bidireccional=False)	Agrega X aristas ponderadas, además especificando si los caminos deberían

	ser o no bidireccionales. Por defecto, no lo son.
--	---

3. Estructuras de Datos Implementadas

3.1 Tabla Hash

Implementación de una tabla hash propia según lo especificado. Usa hashing polinomial (base 31) sobre la representación en bytes de la clave, maneja las colisiones por encadenamiento, y hace rehashing automático cuando el factor de carga supera 0.7, redimensionando siempre a un tamaño primo (mejor distribución con el hash polinomial). La tabla en sí soporta insertar, buscar, borrar, rehash, calcular factor de carga, cubetas ocupadas, máximo tamaño de cubetas, devolver todos, y mostrar un reporte en terminal si es necesario.

Nota: La key debe ser convertible a string. Si la clave ya existía, se actualiza el valor sin contar colisión adicional (actualización, no una inserción nueva); si es una clave nueva en una cubeta ya ocupada, se incrementa el contador de colisiones.

Resultados experimentales (carga de los 500 incidentes del dataset):

Métrica	Valor final
Elementos almacenados	500
Tamaño de tabla final	1361 (primo)
Factor de carga final	0.367
Colisiones totales	43
Cubetas ocupadas	457 de 1361
Tamaño máximo de cubeta	2
Rehashes ejecutados	7

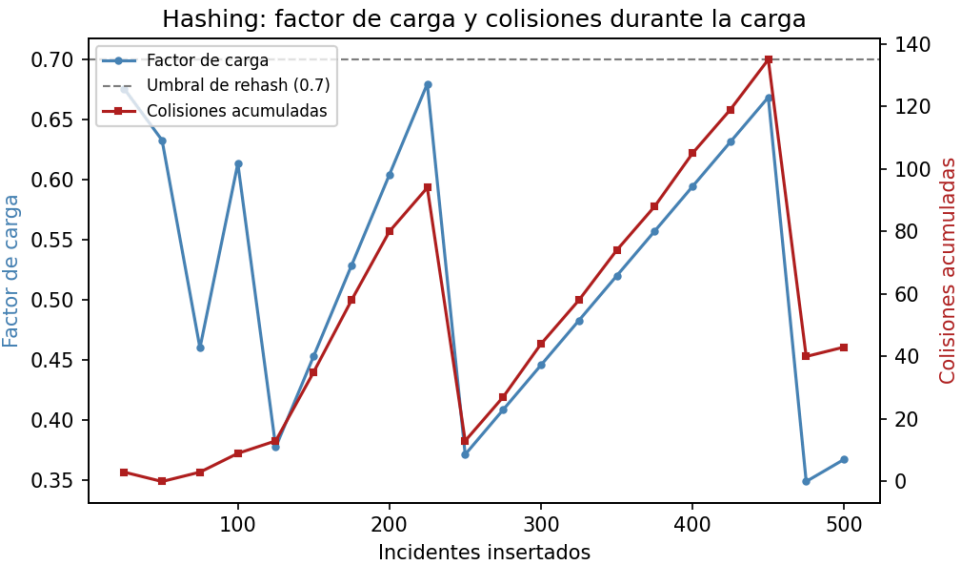


Figura 1. Evolución del factor de carga y colisiones acumuladas durante la carga de 500 incidentes. Las caídas en el factor de carga corresponden a los 7 rehashes disparados automáticamente.

Se ve una distribución razonablemente uniforme. El rehashing automático mantuvo el factor de carga siempre por debajo del umbral de 0.7, evitando la degradación de las operaciones a $O(n)$.

Notar que la tabla fue iniciada con valores mínimos al principio (tamaño=8), aunque se supiera que debían ser por lo menos 500 incidentes. Esto se hizo para reflejar mejor cómo se comporta esta tabla en un escenario “real”, siguiendo el ejemplo de implementaciones comunes de las librerías.

3.2 Priority Queue

Implementada como max-heap binario sobre un arreglo, sin usar heapq de Python (No es especificado que se deba implementar de esta manera, pero se hizo así para mantener el “estilo” con respecto a la implementación de la tabla hash).

La prioridad de cada incidente se calcula como $\text{prioridad} = \text{severidad} \times \text{factor_tiempo}$, donde factor_tiempo crece con las horas transcurridas desde el reporte, lo que evita que un incidente no sea resuelto en ciertos casos donde lleguen incidentes con mayor prioridad “cruda” antes de que se pueda pasar a este.

Se mantiene un diccionario auxiliar pos de los incidentes para que `priorityUpdate` pueda localizar cualquier incidente en $O(1)$ dentro del heap en vez de recorrer el todo en $O(n)$.

Operación	Complejidad
<code>insert(incident)</code>	$O(\log n)$
<code>extractUrgent()</code>	$O(\log n)$
<code>priorityUpdate(id)</code>	$O(\log n) = \text{localización } O(1) + \text{reposicionamiento } O(\log n)$
<code>topKCrit(k)</code>	$O(k \log n)$
<code>calcPriority(incident)</code>	$O(1)$

Resultados experimentales (500 incidentes):

Operación	Tiempo total	Tiempo promedio por operación
Inserción (500 incidentes)	0.80 ms	1.6 μ s
Extracción (500 incidentes)	1.23 ms	2.46 μ s

3.3 Grafo

Estructura genérica de grafo ponderado dirigido independiente de los ADTs, reutilizada tanto por RoadNetwork como por los algoritmos de búsqueda.

Sobre los nombres y la normalización:

Los nodos se identifican por nombre como "Centro_A", "Zona_17", etc. no por coordenadas, ya que refleja mejor el input real de un despachador. El problema de esto es que introduce el riesgo de que "Zona 1", "zona_1" y "ZONA_1" se traten como nodos distintos- por eso la solución adoptada fue centralizar la normalización (strip + minúsculas + guión bajo) dentro de la clase graph misma en `agregarNodo`, `agregarArista`, `vecinos` y `coordenadas`; en vez de exigir que cada punto de entrada del sistema recuerde normalizar. Esto garantiza que ningún nodo pueda entrar sin normalizar, sin importar desde qué parser o módulo se cargue.

Operación	Complejidad
<code>agregarNodo(nodo)</code>	$O(1)$ promedio

agregarArista(origen, destino, peso)	$O(1)$ promedio
vecinos(nodo)	$O(1)$
obtenerNodos()	$O(V)$
obtenerAristas()	$O(V + E)$

4. Algoritmos de Ordenamiento

Se implementaron MergeSort y QuickSort de forma genérica para reusar el mismo código con incidentes por tiempo, por prioridad, zonas por frecuencia, etc.

Algoritmo	Complejidad promedio	Peor caso	Espacio extra	Estable
MergeSort	$O(n \log n)$	$O(n \log n)$	$O(n)$	Sí
QuickSort (pivote aleatorio)	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No

QuickSort usa selección de pivote aleatoria en cada partición. Esto ya que en un dataset ya ordenado o casi ordenado se da el peor caso $O(n^2)$. Con pivote aleatorio se mantiene el caso esperado $O(n \log n)$ sin importar el orden de entrada.

Resultados experimentales (ordenando incidentes por timestamp, muestras aleatorias del dataset):

n	MergeSort (ms)	QuickSort (ms)
50	0.101	0.058
100	0.152	0.101
250	0.418	0.372
500	0.977	0.744

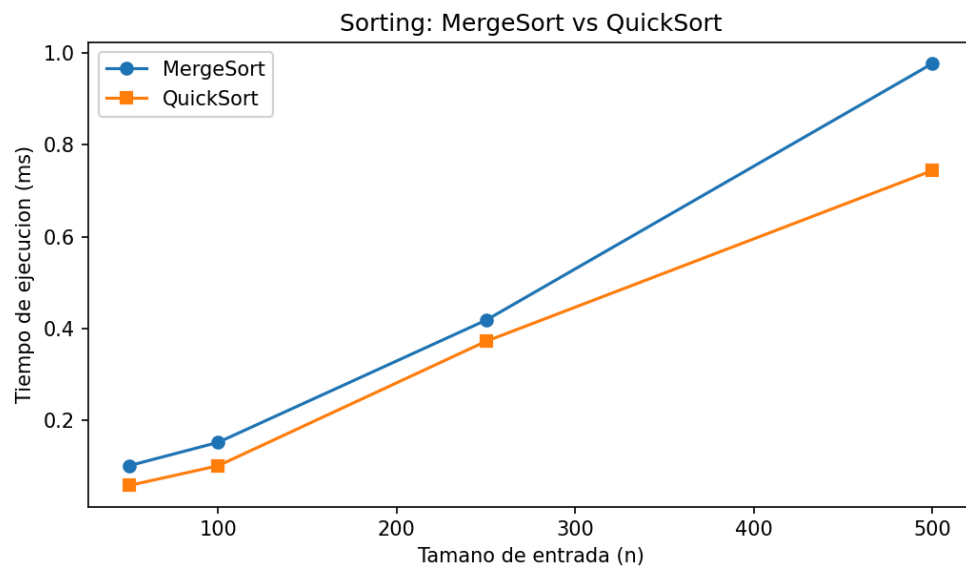


Figura 2. Tiempo de ejecución de MergeSort y QuickSort para distintos tamaños de entrada.

QuickSort resultó consistentemente más rápido que MergeSort en este rango de n , lo esperado dado que MergeSort siempre incurre en el costo de asignar arreglos auxiliares en cada nivel de recursión ($O(n)$ espacio extra), mientras que QuickSort particiona “in-place”.

5. Algoritmos de Búsqueda

Se implementaron los dos algoritmos obligatorios (BFS y UCS) más la extensión opcional (A^*), sobre la red vial de 50 nodos y 100 aristas.

5.1 BFS

Encuentra el camino con menor cantidad de saltos (aristas), sin considerar el peso. Complejidad $O(V + E)$.

5.2 UCS

Encuentra el camino de menor costo acumulado usando una cola de prioridad. Complejidad $O((V + E) \log V)$.

5.3 A^* con heurística Haversine

A^* prioriza la frontera de búsqueda por $\text{costo_real} + \text{heurística}(\text{nodo})$, en vez de solo costo_real , guiando la exploración hacia el destino. La heurística implementada calcula la distancia geográfica real entre nodos (fórmula de Haversine, apropiada para coordenadas lat/lon esféricas) y la convierte a un tiempo estimado asumiendo la velocidad máxima plausible de la red de 120 km/h.

Resultados experimentales:

Algoritmo	Nodos visitados (prom.)	Mín. / Máx.	Tiempo promedio
BFS	40.33	12 / 50	20.6 μs
UCS	24.90	3 / 50	45.2 μs
A^*	19.47	2 / 50	105.9 μs

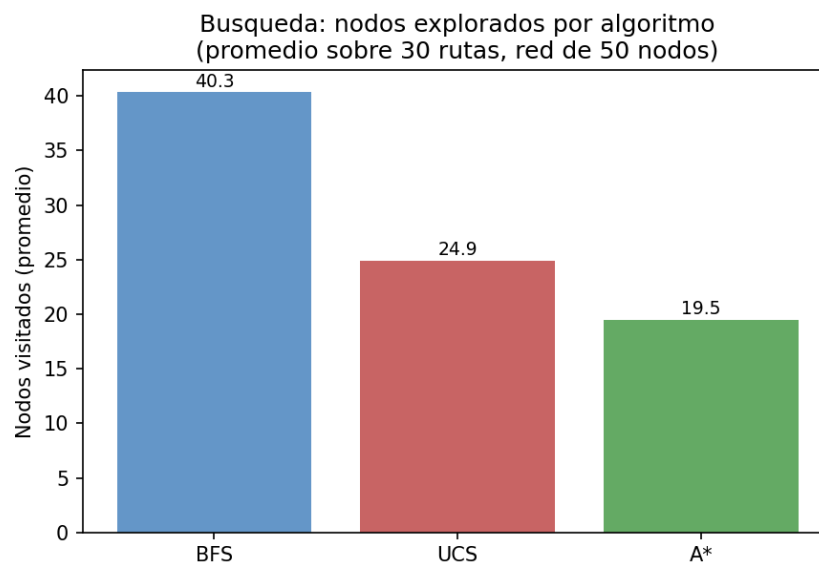


Figura 3. Nodos explorados en promedio por cada algoritmo de búsqueda, sobre 30 rutas distintas en la misma red de 50 nodos.

BFS explora en promedio más del doble de nodos que A*, ya que ignora por completo el costo y expande por niveles sin ninguna guía. UCS mejora sobre BFS porque prioriza por costo real, pero sigue explorando en todas las direcciones por igual. A* explora consistentemente menos nodos que ambos gracias a la heurística geográfica, que descarta tempranamente direcciones que se alejan del destino.

Es de notar que mientras que A* explora menos nodos- en si toma más tiempo que las demás para estas pruebas. Esto se debe a tener que calcular la fórmula para los incidentes. De todas formas, esto sería compensado con respecto al tamaño del dataset. Mientras más grande es, mas efectivo seria A*.

6. Escenario Integrado

Se cargaron los 500 incidentes en la red, extrayendo repetidamente el incidente más urgente de la cola de prioridad, para luego encontrar el centro de emergencia más cercano mediante A* y calcular la ruta con el costo total hasta agotar la cola.

Métrica	Resultado
Incidentes procesados	500
Atendidos con ruta encontrada	500 (100%)
Sin ruta disponible	0
Costo (tiempo de viaje) promedio	12.62 minutos
Costo mínimo	0.60 minutos
Costo máximo	42.90 minutos

7. Notas y Decisiones de Diseño Relevantes

7.1 Alcance de la restricción "no usar dict"

El enunciado sólo prohíbe explícitamente el uso de dict de Python únicamente en la Parte 2, donde la tabla hash es el objeto de estudio. En el resto del proyecto se usan dict y set nativos de Python como estructuras auxiliares (por ejemplo en la lista de adyacencia del grafo). Esta decisión se tomó dado que la restricción no es mencionada en las otras partes del enunciado, logrando evitar dependencias cruzadas innecesarias entre módulos y además agregar complejidad al hacer de la tabla hash tomar valores fuera de objetos incidentes.

7.2 heapq de la biblioteca estándar en los algoritmos de búsqueda

UCS y A* usan heapq en lugar del pHeap propio. Esto principalmente porque pHeap fue diseñado específicamente para incidentes, no para pares genéricos. Forzar pHeap a este propósito habría requerido generalizar más allá de su responsabilidad original sin necesidad, ya que el enunciado no exige una implementación propia de heap para esta sección.

7.3 Heurística de A*

El código que representa el cálculo matemático como tal fue obtenido por una combinación de fuentes online e IA. Principalmente porque, siendo honesto, no soy tan familiar con los cálculos que se ocupan en situaciones más "reales", entonces para evitar errores debido a esto se optó por buscar soluciones ya comprobadas.

7.4 Sobre el uso de IA en general

Para ser transparente ante este tema, IA efectivamente fue ocupada para el desarrollo de este proyecto- pero no para la generación completa del código. Sino que fue ocupada en:

- Generación de código para benchmark.
- Generación de datos de prueba.
- Generación de tablas respecto a los datos obtenidos.
- Debugging.
- En parte, la heurística de A* antes mencionada.
- Consultas sobre TKinter y Matplotlib.

El diseño y código principal fue hecho propiamente. Notar que el código relacionado con benchmarks además de los datos generados fueron revisados que siguieran los parámetros definidos.

7.5 Imágenes del programa

Para evitar la extensión innecesaria de este documento, se evitó adjuntar imágenes del programa aquí mismo. Se pueden ver como tal en el repositorio de Github.

7.6 Uso de botones

El programa cuenta con las opciones de “Reportar Incidente”, “Atender Siguiente”, “Iniciar Modo Automático”, y “Ver Historial”. Por claridad, estos son sus propósitos:

Botón	Propósito
Reportar Incidente	Añade un (1) incidente de manera manual. El usuario tiene que especificar los datos de este que aparecen en la ventana. NOTA: El ID no es verificado si ya existe en la tabla hash- si es que efectivamente existe, entonces el incidente anterior será actualizado por este.
Atender Siguiente	“Atiende” el siguiente incidente que corresponda. Mostrando datos como los caminos y otros.
Iniciar Modo Automático	Igual que “Atender Siguiente”, pero lo hace de manera automática hasta que no queden incidentes que están sin atender.
Ver Historial	Muestra el historial de los incidentes, su estado, urgencia, etc. El usuario puede ordenar según opciones propuestas ahí mismo.

7.7 Lectura de archivos y Ejecución del programa

Requiere de los archivos CSV o JSON apropiados. En teoría no importa cual, pero el programa fue testeado principalmente con CSV (los cuales igual están en el repositorio de github). El orden de que archivo se carga primero es mencionado en el README del repositorio.

Para la lectura de los archivos como tal, se ocupó la librería JSON y Pandas. Esto es manejado por los parsers correspondientes en la carpeta FileHandlers.

8. Conclusiones

El sistema implementado cubre la totalidad de los requerimientos obligatorios del enunciado (ADTs, tabla hash, heap, dos algoritmos de ordenamiento, grafos, BFS y UCS) más la extensión opcional de A* con heurística real, validado “end-to-end” sobre un dataset de 500 incidentes, 50 nodos y 100 aristas.

Los resultados experimentales confirman el comportamiento asintótico esperado de cada estructura: crecimiento logarítmico en las operaciones del heap, factor de carga controlado mediante rehashing automático en la tabla hash, la ventaja constante de QuickSort sobre MergeSort, y una reducción medible en nodos explorados de A* frente a UCS y BFS gracias a la heurística geográfica.